

Week 7 Assignment Summary

Incremental Data Processing with Delta Lake — Employee Dataset (Databricks)

Objective

Implement an incremental data processing pipeline on Databricks using Delta Lake: load an employee master dataset into a Delta table, apply an incremental batch via a **MERGE** (upsert) operation to update existing employee records and insert new ones, query the merged results with Spark SQL, and inspect the Delta transaction log for versioning and auditability.

Environment

Platform	Databricks (Spark SQL / PySpark, Delta Lake)
Runtime	Databricks Runtime 18.x (aarch64, Photon, Scala 2.13)
Catalog / Schema	workspace.default
Source tables	employee_incremental_week_7, employee_master_week_7
Target Delta table	workspace.default.employee_delta

Pipeline Steps

- 1 **Load source tables:** two employee tables were read into DataFrames — a 10-row batch (seed/base data: employees E103, E107, E112, E118, E120–E125) and a 20-row batch (employees E101–E120, containing both overlapping and net-new employee IDs).
- 2 **Write base Delta table:** the 10-row batch was written as `workspace.default.employee_delta` using `.format("delta").mode("overwrite").saveAsTable(...)`.
- 3 **MERGE (upsert):** using `DeltaTable.forName()` and `.merge()`, the 20-row batch was merged into the base table on `employee_id`, with `whenMatchedUpdateAll()` and `whenNotMatchedInsertAll()`.
- 4 **Query results:** the merged table was registered as a temp view (`employees`) and queried with Spark SQL — full listing ordered by `employee_id`, a filtered lookup by specific IDs, and a range filter (`employee_id >= 'E121'`).
- 5 **Audit the transaction log:** `deltaTable.history()` was inspected to review every version of the table (CREATE, MERGE, and Databricks' auto-OPTIMIZE compactions) along with row-level merge metrics.
- 6 **Time travel & maintenance:** a versioned read (`option("versionAsOf", 0)`) and a `VACUUM ... RETAIN 168 HOURS` command were run to demonstrate historical reads and safe cleanup of stale data files.

MERGE Results

Base table before merge: **10 rows**. Incremental source: **20 rows**, of which **5 employee IDs already existed** in the base table (E103, E107, E112, E118, E120) and **15 were new**. The MERGE correctly updated the 5 matches (e.g. E103's status changed Active → Inactive and location Mumbai → Pune) and inserted the 15 new employees, bringing the table to its final size.

Metric	Value
Rows in base/seed table (pre-merge)	10
Rows in incremental source batch	20
Matched rows (UPDATED)	5
Unmatched rows (INSERTED)	15
Final row count (post-merge)	25

Example: Updated Record

Employee **E103 (Rohan Gupta)** existed in the base table as Finance / Mumbai / Active. After the MERGE, the incremental source's version overwrote it to Finance / Pune / **Inactive** — confirming `whenMatchedUpdateAll()` replaced the full row rather than only inserting a duplicate.

Delta Transaction Log (History)

`deltaTable.history()` returned **7 versions**, showing that the pipeline was executed end-to-end **three times** during development. Databricks automatically compacts small files after each write (the interleaved OPTIMIZE versions), which is normal auto-optimize behavior, not a manual step in the notebook.

Ver.	Operation	Key metrics
0	CREATE OR REPLACE TABLE AS SELECT	10 rows written (base table seeded)
1	MERGE	5 matched/updated, 15 inserted, 20 source rows → 1st run
2	OPTIMIZE (auto)	File compaction after v1
3	MERGE	20 matched/updated, 0 inserted → 2nd run (all already present)
4	OPTIMIZE (auto)	File compaction after v3
5	MERGE	20 matched/updated, 0 inserted → 3rd run (idempotent)
6	OPTIMIZE (auto)	File compaction after v5

Observation: re-running the MERGE cell a second and third time correctly reported 20 matched updates and 0 inserts each time — demonstrating that Delta Lake's MERGE is **idempotent**: replaying the same incremental batch does not create duplicate rows.

Key Concepts Demonstrated

- Writing a DataFrame as a managed Delta table (`saveAsTable`).
- Programmatic upserts with `DeltaTable.merge()`, `whenMatchedUpdateAll()`, `whenNotMatchedInsertAll()`.

- Querying Delta tables with Spark SQL via temp views.
- Auditing changes through the Delta transaction log (`DESCRIBE HISTORY / .history()`).
- Time travel reads with `versionAsOf`.
- Table maintenance with `VACUUM ... RETAIN 168 HOURS` to safely remove stale files while preserving the retention window needed for time travel.

Conclusion

This week's exercise covered the core incremental-load pattern used in real-world lakehouse pipelines: seed a Delta table, merge in new/changed records with a single atomic `MERGE` statement, verify correctness via SQL queries and row counts, and use Delta's built-in versioning for auditability and recovery. The repeated `MERGE` runs additionally confirmed the operation's idempotency, an important property for reliable, re-runnable data pipelines.